

## Ficha 2

# Ferramentas De Análise De Performance (Continuação) & Emulação/Geração De Tráfego De Diferentes Tipos De Aplicações

Trabalho Elaborado por:

Diogo Fernando Águia Costa (8240696@estg.ipp.pt)

Curso de:

LSIRC - Licenciatura Em Segurança Informática Em Redes De Computadores

Disciplina de:

PPR - Projeção e Planeamento de Redes

Docentes:

João Magalhães (jpm@estg.ipp.pt)

Hélio Sousa (hcs@estg.ipp.pt)

Felgueiras, 25 de março de 2026

# Conteúdo

|          |                                                                                   |           |
|----------|-----------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Exercício 1: Ferramentas de Análise de Performance</b>                         | <b>7</b>  |
| 1.0.1    | Preparação do Ambiente de Testes . . . . .                                        | 7         |
| 1.1      | Exercício 1.1 . . . . .                                                           | 8         |
| 1.1.1    | Metodologia Inicial (Rede Wi-Fi) . . . . .                                        | 8         |
| 1.1.2    | Adaptação da Topologia (Rede Interna) . . . . .                                   | 9         |
| 1.1.3    | Apresentação e Análise de Resultados . . . . .                                    | 10        |
| 1.1.4    | Análise Comparativa com o protocolo iperf . . . . .                               | 11        |
| 1.2      | Exercício 1.2 . . . . .                                                           | 11        |
| 1.2.1    | Metodologia de Sobrecarga de Rede . . . . .                                       | 11        |
| 1.2.2    | Resultados Sob Carga e Discussão . . . . .                                        | 12        |
| 1.3      | Exercício 1.3 . . . . .                                                           | 13        |
| 1.3.1    | Introdução à Ferramenta Pathload . . . . .                                        | 13        |
| 1.3.2    | Execução de Testes . . . . .                                                      | 14        |
| <b>2</b> | <b>Exercício 2: Emulação/Geração De Tráfego De Diferentes Tipos De Aplicações</b> | <b>17</b> |
| 2.1      | Exercício 2.1 . . . . .                                                           | 17        |
| 2.1.1    | Introdução . . . . .                                                              | 18        |
| 2.1.2    | Configuração do Teste . . . . .                                                   | 18        |
| 2.1.3    | Metodologia do Teste . . . . .                                                    | 20        |
| 2.1.4    | Resultados Obtidos . . . . .                                                      | 20        |
| 2.2      | Exercício 2.2 . . . . .                                                           | 22        |
| 2.2.1    | Introdução . . . . .                                                              | 23        |
| 2.2.2    | SIP . . . . .                                                                     | 23        |
| 2.2.3    | DDoS . . . . .                                                                    | 25        |

|       |                                  |    |
|-------|----------------------------------|----|
| 2.3   | Exercício 2.3 . . . . .          | 30 |
| 2.3.1 | Introdução . . . . .             | 30 |
| 2.3.2 | Início dos testes . . . . .      | 31 |
| 2.3.3 | Resultados Obtidos . . . . .     | 32 |
| 2.3.4 | Análise dos Resultados . . . . . | 33 |
| 2.3.5 | Conclusão . . . . .              | 33 |

# Lista de Figuras

|      |                                                                                         |    |
|------|-----------------------------------------------------------------------------------------|----|
| 1.1  | Clonagem do repositório do Pathrate . . . . .                                           | 8  |
| 1.2  | Inicialização da ferramenta no Servidor . . . . .                                       | 8  |
| 1.3  | Inicialização da ferramenta no Cliente . . . . .                                        | 8  |
| 1.4  | Erro originado por excesso de medições ignoradas ( <i>Packet Dispersion</i> ) . . . . . | 9  |
| 1.5  | Estimativa de capacidade total utilizando o Pathrate em Rede Interna . . . . .          | 10 |
| 1.6  | Teste de débito útil máximo em UDP com a ferramenta iperf . . . . .                     | 11 |
| 1.7  | Aquisição do ficheiro de teste . . . . .                                                | 11 |
| 1.8  | inicialização do serviço daemon do SSH . . . . .                                        | 12 |
| 1.9  | Geração de carga via SCP . . . . .                                                      | 12 |
| 1.10 | Execução do Pathrate sob intenso congestionamento de rede . . . . .                     | 12 |
| 1.11 | Clonagem do repositório do Pathload . . . . .                                           | 13 |
| 1.12 | Configuração do Pathload com flags de compatibilidade do compilador . . . . .           | 14 |
| 1.13 | Execução do comando make e geração dos binários executáveis . . . . .                   | 14 |
| 1.14 | Resultado do teste Pathload sem tráfego concorrente . . . . .                           | 14 |
| 1.15 | Resultado do teste Pathload sob carga . . . . .                                         | 15 |
| 2.1  | Tabela de parâmetros do gerador de tráfego (packETH/Ostinato) . . . . .                 | 18 |
| 2.2  | Configuração da largura de banda no Ostinato . . . . .                                  | 18 |
| 2.3  | Configuração do envio em ciclo infinito . . . . .                                       | 19 |
| 2.4  | Configuração dos protocolos e tamanho do payload . . . . .                              | 19 |
| 2.5  | Comando de inicialização da captura de pacotes com o tcpdump . . . . .                  | 20 |
| 2.6  | Comando de análise do ficheiro de captura com o tcptrace . . . . .                      | 20 |
| 2.7  | Resultados estatísticos da captura de tráfego (tcptrace) . . . . .                      | 21 |
| 2.8  | Protocol Selection(SIP) . . . . .                                                       | 23 |

|      |                                                                                    |    |
|------|------------------------------------------------------------------------------------|----|
| 2.9  | Protocol Selectio(SIP)                                                             | 24 |
| 2.10 | Escutar a interface eth0                                                           | 24 |
| 2.11 | Resultados Obtidos no Teste SIP                                                    | 25 |
| 2.12 | Configuração da camada de transporte (L4) para simulação de SYN Flood no Ostinato. | 25 |
| 2.13 | Configuração das flags TCP para envio de pacotes SYN.                              | 26 |
| 2.14 | Configuração de sobreposição (Override) de parâmetros para o SYN Flood.            | 26 |
| 2.15 | Configuração da camada de rede para o protocolo ICMP no Ostinato.                  | 26 |
| 2.16 | Configuração da mensagem ICMP do tipo Echo Request.                                | 27 |
| 2.17 | Configuração da camada de transporte para o protocolo UDP no Ostinato.             | 27 |
| 2.18 | Configuração de sobreposição (Override) de parâmetros para o UDP Flood.            | 27 |
| 2.19 | Comando de captura de tráfego na interface eth0 utilizando o tcpdump.              | 28 |
| 2.20 | Comando de visualização de estatísticas de pacotes capturados com o tshark.        | 28 |
| 2.21 | Comando de análise detalhada do ficheiro pcap gerado.                              | 28 |
| 2.22 | Estatísticas globais do tráfego capturado durante o ataque DDoS.                   | 28 |
| 2.23 | Análise detalhada dos pacotes do ataque DDoS capturados.                           | 29 |
| 2.24 | Inicialização da interface interativa da ferramenta Scapy.                         | 30 |
| 2.25 | Configuração da interface eth0 em modo de escuta no servidor.                      | 31 |
| 2.26 | Instrução do Scapy utilizada no cliente para envio de pacotes.                     | 31 |
| 2.27 | Resultados da captura de pacotes efetuada no servidor através do tshark.           | 32 |
| 2.28 | Análise da conexão TCP gerada, utilizando a ferramenta tcptrace.                   | 32 |

# Lista de Tabelas

|     |                                                                              |    |
|-----|------------------------------------------------------------------------------|----|
| 1.1 | Impacto do congestionamento na estimativa de capacidade . . . . .            | 13 |
| 1.2 | Comparação dos resultados obtidos com o Pathload . . . . .                   | 15 |
| 2.1 | Comparação do fluxo de tráfego entre a máquina de origem e destino . . . . . | 22 |
| 2.2 | Tráfego gerado por intervalo de tempo . . . . .                              | 25 |
| 2.3 | Tráfego gerado por intervalo de tempo no ataque DDoS. . . . .                | 29 |

# Exercício 1: Ferramentas de Análise de Performance

## Enunciado

O *Pathrate* é uma aplicação que disponibiliza um método para determinar a largura de banda total possível entre dois pontos da rede, mesmo quando essa ligação se encontra em carga. Adicionalmente, esta aplicação pode ser utilizada para estimar o débito máximo teórico entre dois pontos, permitindo a identificação de possíveis zonas de estrangulamento (*bottlenecks*) em situações de tráfego intenso.

<https://faculty.cc.gatech.edu/~dovrolis/bw-est/pathrate.html>

<https://faculty.cc.gatech.edu/~dovrolis/bw-est/pathload.html>

<https://github.com/brucespang/pathrate>

## Notas:

- O servidor executa a aplicação `pathrate_snd`, enquanto o cliente executa `pathrate_rcv`.
- Em testes com a rede em carga, a precisão do *Pathrate* está dependente da disponibilidade do CPU, devendo garantir-se que este não se encontra em sobrecarga.

### 1.0.1 Preparação do Ambiente de Testes

Para a utilização da ferramenta *Pathrate*, procedeu-se à obtenção do respetivo código-fonte através do repositório alojado no GitHub, efetuando o *clone* para os ambientes virtuais de ambas as máquinas. Concluída a transferência, procedeu-se à compilação dos ficheiros em linguagem C para gerar os binários executáveis do cliente e do servidor.

```
(machine2@machine2)-[~]
$ git clone https://github.com/brucespang/pathrate
Cloning into 'pathrate'...
remote: Enumerating objects: 39, done.
remote: Total 39 (delta 0), reused 0 (delta 0), pack-reused 39 (from 1)
Receiving objects: 100% (39/39), 79.54 KiB | 21.00 KiB/s, done.
Resolving deltas: 100% (19/19), done.
```

Figura 1.1: Clonagem do repositório do Pathrate

Nos terminais, a máquina com função de servidor instanciará a aplicação `pathrate_snd`, ao passo que a máquina cliente executará a aplicação `pathrate_rcv`.

```
(machine2@machine2)-[~]
$ cd pathrate

(machine2@machine2)-[~/pathrate]
$ gcc pathrate_snd.c pathrate_snd_func.c -o pathrate_snd -lm
```

Figura 1.2: Inicialização da ferramenta no Servidor

```
(kali@kali)-[~]
$ cd pathrate

(kali@kali)-[~/pathrate]
$ gcc pathrate_rcv.c pathrate_rcv_func.c -o pathrate_rcv -lm
```

Figura 1.3: Inicialização da ferramenta no Cliente

## 1.1 Exercício 1.1

### Enunciado

Execute um teste para estimar o débito máximo disponível entre uma VM no seu computador e a VM no computador de um colega seu. Faça dois testes: um para a rede cablada e outro para a rede sem fios. Tenha em atenção o parâmetro “Resolution” que aparece no ecrã. Interprete os resultados e compare-os com os obtidos com outras aplicações (e.g., `iperf`, `netperf`, ...).

### 1.1.1 Metodologia Inicial (Rede Wi-Fi)

De acordo com os requisitos, deveriam ser executados testes em rede Wi-Fi e rede cablada. Dada a impossibilidade técnica de interligar as máquinas fisicamente por cabo Ethernet, o escopo focou-se na ligação sem fios.

A arquitetura inicial assentou nas seguintes configurações:

- **Servidor:** machine2 (IP: 192.168.1.254)
- **Cliente:** Kali (IP: 192.168.1.252)



Durante a execução na rede sem fios, verificaram-se constrangimentos severos, culminando sistematicamente na incapacidade de concluir a Fase I do teste, originando o erro ilustrado abaixo.

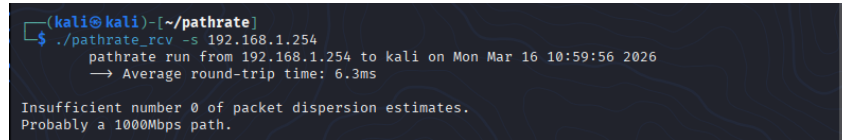
A terminal window with a dark background. The prompt is '(kali@kali)-[~/pathrate]'. The command './pathrate\_rcv -s 192.168.1.254' has been executed. The output shows 'pathrate run from 192.168.1.254 to kali on Mon Mar 16 10:59:56 2026' and '→ Average round-trip time: 6.3ms'. Below this, an error message is displayed: 'Insufficient number 0 of packet dispersion estimates. Probably a 1000Mbps path.'

Figura 1.4: Erro originado por excesso de medições ignoradas (*Packet Dispersion*)

A mensagem *Too many ignored measurements* comprova que o *Pathrate* não conseguiu recolher amostras estatísticas consistentes da dispersão de pacotes. Este bloqueio é característico de redes Wi-Fi partilhadas com máquinas virtuais, onde a variação abrupta de latência (*jitter*) e a perda de datagramas UDP impedem a convergência do algoritmo de medição.

### 1.1.2 Adaptação da Topologia (Rede Interna)

Perante a inexequibilidade de obter medições fiáveis sobre a rede sem fios, a topologia foi readaptada. Optou-se por alojar ambas as máquinas virtuais num único *host* físico, interligando-as através de um comutador virtual isolado (*Internal Network*). Esta abordagem suprime a interferência do meio físico partilhado, garantindo condições laboratoriais controladas.

As configurações de endereçamento foram atualizadas para:

- **Servidor (machine2):** IP 192.168.10.10
- **Cliente (kali):** IP 192.168.10.20

### 1.1.3 Apresentação e Análise de Resultados

```
(kali@kali)-[~/pathrate]
$ ./pathrate_rcv -s 192.168.10.10
pathrate run from 192.168.10.10 to kali on Mon Mar 16 12:33:09 2026
→ Average round-trip time: 0.8ms

→ Capacity Resolution: 10.2 Mbps

-- Phase I: Detect possible capacity modes --

→ Train length: 2 - Packet size: 600B → 0% completed
→ Train length: 3 - Packet size: 898B → 2% completed
→ Train length: 4 - Packet size: 1196B → 5% completed
→ Train length: 5 - Packet size: 1488B → 7% completed
→ Train length: 5 - Packet size: 1488B → 10% completed
→ Train length: 6 - Packet size: 1488B → 12% completed
→ Train length: 6 - Packet size: 1488B → 15% completed
→ Train length: 6 - Packet size: 1488B → 17% completed
→ Train length: 6 - Packet size: 1488B → 20% completed
→ Train length: 6 - Packet size: 1488B → 22% completed
→ Train length: 7 - Packet size: 1488B → 25% completed
→ Train length: 7 - Packet size: 1488B → 27% completed
→ Train length: 7 - Packet size: 1488B → 30% completed
→ Train length: 7 - Packet size: 1488B → 32% completed
→ Train length: 7 - Packet size: 1488B → 35% completed
→ Train length: 7 - Packet size: 1488B → 37% completed
→ Train length: 7 - Packet size: 1488B → 40% completed
→ Train length: 7 - Packet size: 1488B → 42% completed
→ Train length: 8 - Packet size: 1488B → 45% completed
→ Train length: 9 - Packet size: 1488B → 47% completed
→ Train length: 9 - Packet size: 1488B → 50% completed
→ Train length: 9 - Packet size: 1488B → 52% completed
→ Train length: 10 - Packet size: 1488B → 55% completed
→ Train length: 10 - Packet size: 1488B → 57% completed
→ Train length: 10 - Packet size: 1488B → 62% completed
→ Train length: 10 - Packet size: 1488B → 65% completed
→ Train length: 10 - Packet size: 1488B → 67% completed
→ Train length: 10 - Packet size: 1488B → 70% completed
→ Train length: 10 - Packet size: 1488B → 72% completed
→ Train length: 10 - Packet size: 1488B → 75% completed
→ Train length: 10 - Packet size: 1488B → 77% completed
→ Train length: 10 - Packet size: 1488B → 80% completed
→ Train length: 10 - Packet size: 1488B → 82% completed
→ Train length: 10 - Packet size: 1488B → 85% completed
→ Train length: 10 - Packet size: 1488B → 87% completed
→ Train length: 10 - Packet size: 1488B → 90% completed
→ Train length: 10 - Packet size: 1488B → 92% completed
→ Train length: 11 - Packet size: 1488B → 95% completed
→ Train length: 11 - Packet size: 1488B → 97% completed

-- Local modes : In Phase I --

-- Local modes : In Phase I --
39 Mbps 39 Mbps 49 Mbps - 304 measurements
Modal bell: 749 measurements - low : 7.1 Mbps - high : 157 Mbps

-- Phase II: Estimate Asymptotic Dispersion Rate (ADR) --

-- Number of trains: 500 - Train length: 48 - Packet size: 1488B
```

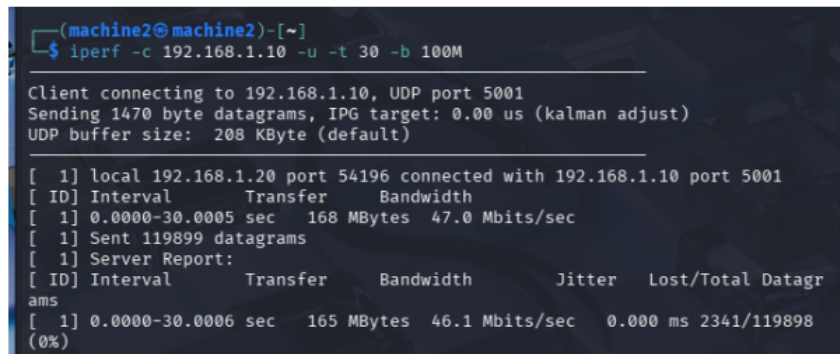
Figura 1.5: Estimativa de capacidade total utilizando o Pathrate em Rede Interna

Com a transição para a rede interna, a aplicação executou sem anomalias, estabilizando o tempo de ida e volta em apenas **0.8 ms**. O parâmetro de resolução foi estabelecido em 10.2 Mbps.

O *Pathrate* opera enviando "comboios" de pacotes variando o seu comprimento e tamanho para estimular as filas de espera da ligação. Ao contrário do cenário Wi-Fi, a **Fase I** foi concluída com êxito. A ferramenta conseguiu detetar padrões locais de capacidade sustentados, agrupando 304 medições consistentes no intervalo restrito de **39 Mbps a 49 Mbps**.

A **Fase II** procedeu à avaliação da Taxa de Dispersão Assintótica (ADR) através do envio de 500 comboios de 48 pacotes (1488 bytes/pacote). O facto de o teste não ter abortado garante que a estimativa final reflete a capacidade real do estrangulamento da interface virtual, situando-se nos valores apresentados na curva modal.

### 1.1.4 Análise Comparativa com o protocolo iperf



```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -u -t 30 -b 100M

Client connecting to 192.168.1.10, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 192.168.1.20 port 54196 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-30.0005 sec  168 MBytes  47.0 Mbits/sec
[ 1] Sent 119899 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer    Bandwidth      Jitter    Lost/Totl Datagr
ams
[ 1] 0.0000-30.0006 sec  165 MBytes  46.1 Mbits/sec  0.000 ms 2341/119898
(0%)
```

Figura 1.6: Teste de débito útil máximo em UDP com a ferramenta iperf

Para contextualizar a estimativa, estabeleceu-se uma comparação retroativa com a ferramenta *iperf* (dados obtidos na Ficha 1). Enquanto o *Pathrate* procura descobrir o limite de capacidade bruta da infraestrutura através da análise temporal de dispersão, o *iperf* foca-se em saturar a rede com carga UDP útil para medir o débito (*throughput*) efetivamente processado pela camada de transporte.

Os dados corroboram a fiabilidade de ambos os métodos. O *iperf* registou um débito efetivo contínuo de **46.1 Mbits/s**. Este valor insere-se de forma exata e coerente no modo principal de capacidade bruta detetado pelo *Pathrate* (entre os **39 Mbps e 49 Mbps**), validando a precisão da medição da arquitetura virtual.

## 1.2 Exercício 1.2

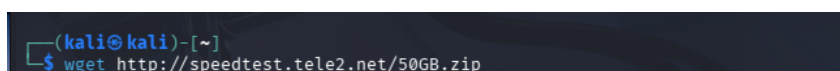
### Enunciado

Execute novamente o teste anterior, mas agora origine tráfego (*downloads* paralelos, etc.) a partir da sua máquina, de forma a carregar a ligação (*link*). Tenha em atenção o parâmetro “Resolution” que aparece no ecrã. Compare os resultados com os obtidos na questão 1.1.

### 1.2.1 Metodologia de Sobrecarga de Rede

Para induzir carga no canal de comunicação durante a medição, recorreu-se ao utilitário SCP (*Secure Copy Protocol*). Foi efetuada a transferência em segundo plano de um ficheiro comprimido de grande dimensão (10 GB), assegurando a saturação sustentada da largura de banda.

A obtenção do ficheiro padrão foi realizada via utilitário *wget*, conectando a um servidor de *Speedtest*:



```
(kali@kali)-[~]
$ wget http://speedtest.tele2.net/50GB.zip
```

Figura 1.7: Aquisição do ficheiro de teste

Após a inicialização do serviço *daemon* do SSH no servidor alvo, despoletou-se a transferência massiva, procedendo-se de imediato à execução paralela do *Pathrate*.

```
(machine2@machine2)-[~/pathrate]
$ sudo systemctl start ssh
```

Figura 1.8: inicialização do serviço daemon do SSH

```
(kali@kali)-[~/Desktop]
$ scp 10GB.zip machine2@192.168.10.10:/home/machine2/Desktop/
```

Figura 1.9: Geração de carga via SCP

### 1.2.2 Resultados Sob Carga e Discussão

```
(kali@kali)-[~/pathrate]
$ ./pathrate_rcv -s 192.168.10.10
pathrate run from 192.168.10.10 to kali on Tue Mar 17 12:40:09 2026
→ Average round-trip time: 1.2ms
→ Capacity Resolution: 5.2 Mbps

-- Phase I: Detect possible capacity modes --
→ Train length: 2 - Packet size: 600B → 0% completed
→ Train length: 3 - Packet size: 898B → 2% completed
→ Train length: 3 - Packet size: 921B → 5% completed
→ Train length: 4 - Packet size: 1219B → 7% completed
→ Train length: 4 - Packet size: 1242B → 10% completed
→ Train length: 4 - Packet size: 1265B → 12% completed
→ Train length: 5 - Packet size: 1488B → 15% completed
→ Train length: 5 - Packet size: 1488B → 17% completed
→ Train length: 5 - Packet size: 1488B → 20% completed
→ Train length: 5 - Packet size: 1488B → 22% completed
→ Train length: 6 - Packet size: 1488B → 25% completed
→ Train length: 6 - Packet size: 1488B → 27% completed
→ Train length: 7 - Packet size: 1488B → 30% completed
→ Train length: 7 - Packet size: 1488B → 32% completed
→ Train length: 7 - Packet size: 1488B → 35% completed
→ Train length: 7 - Packet size: 1488B → 37% completed
→ Train length: 7 - Packet size: 1488B → 40% completed
→ Train length: 7 - Packet size: 1488B → 42% completed
→ Train length: 7 - Packet size: 1488B → 45% completed
→ Train length: 7 - Packet size: 1488B → 47% completed
→ Train length: 7 - Packet size: 1488B → 50% completed
→ Train length: 7 - Packet size: 1488B → 52% completed
→ Train length: 7 - Packet size: 1488B → 55% completed
→ Train length: 7 - Packet size: 1488B → 57% completed
→ Train length: 7 - Packet size: 1488B → 60% completed
→ Train length: 7 - Packet size: 1488B → 62% completed
→ Train length: 7 - Packet size: 1488B → 65% completed
→ Train length: 7 - Packet size: 1488B → 67% completed
→ Train length: 7 - Packet size: 1488B → 70% completed
→ Train length: 7 - Packet size: 1488B → 72% completed
→ Train length: 7 - Packet size: 1488B → 75% completed
→ Train length: 7 - Packet size: 1488B → 77% completed
→ Train length: 7 - Packet size: 1488B → 80% completed
→ Train length: 7 - Packet size: 1488B → 82% completed
→ Train length: 7 - Packet size: 1488B → 85% completed
→ Train length: 7 - Packet size: 1488B → 87% completed
→ Train length: 7 - Packet size: 1488B → 90% completed
→ Train length: 7 - Packet size: 1488B → 92% completed
→ Train length: 7 - Packet size: 1488B → 95% completed
→ Train length: 7 - Packet size: 1488B → 97% completed

-- Local modes : In Phase I --

-- Local modes : In Phase I --
40 Mbps 40 Mbps 45 Mbps - 160 measurements
Modal bell: 786 measurements - low : 4.3 Mbps - high : 88 Mbps

-- Phase II: Estimate Asymptotic Dispersion Rate (ADR) --
-- Number of trains: 500 - Train length: 48 - Packet size: 1488B
```

Figura 1.10: Execução do Pathrate sob intenso congestionamento de rede

A tabela seguinte sistematiza o impacto provocado pelo tráfego de saturação face ao ambiente isolado:

| Métrica ( <i>Pathrate</i> )  | Rede Livre (Ex. 1.1) | Rede Congestionada (Ex. 1.2) |
|------------------------------|----------------------|------------------------------|
| Latência RTT                 | 0.8 ms               | 1.2 ms                       |
| <i>Capacity Resolution</i>   | 10.2 Mbps            | 5.2 Mbps                     |
| Modo de Capacidade Frequente | 39 a 49 Mbps         | 40 a 45 Mbps                 |
| Volume de Medições Válidas   | 304                  | 160                          |
| Amplitude da Curva Modal     | 7.1 Mbps a 157 Mbps  | 4.3 Mbps a 88 Mbps           |

Tabela 1.1: Impacto do congestionamento na estimativa de capacidade

A análise comparativa evidencia de forma clara as reações da ferramenta a um canal ocupado. A latência sofreu um incremento expectável (de 0.8 para 1.2 ms), ilustrando o tempo adicional que os pacotes de medição aguardaram nos *buffers* do comutador virtual, que se encontravam inundados pelos pacotes da transferência SCP.

A *Capacity Resolution* adaptou-se automaticamente de 10.2 para 5.2 Mbps, um ajustamento granular forçado pelo "ruído" estatístico introduzido na rede. Embora a ferramenta mantenha a precisão ao detetar o limite do canal (situando a média nos 40-45 Mbps), a perturbação externa provocou um corte acentuado no volume de medições válidas (redução de quase 50%) e comprimiu de forma severa o teto teórico da curva modal, ilustrando de forma empírica o fenómeno de estrangulamento da capacidade devido a fluxos concorrentes.

## 1.3 Exercício 1.3

### Enunciado

Experimente a aplicação Pathload (`pathload_snd` no servidor, `pathload_rcv` no cliente) e compare os resultados com os obtidos em 1.1 e 1.2.

### 1.3.1 Introdução à Ferramenta Pathload

Para este exercício, será replicado o ambiente de testes dos cenários anteriores, adotando, no entanto, a ferramenta *Pathload*. Enquanto o *Pathrate* procura estimar o limite físico teórico da ligação (capacidade total), o *Pathload* tem como objetivo avaliar a **largura de banda disponível** (*Available Bandwidth*), ou seja, a percentagem do canal de rede que se encontra efetivamente livre e que pode ser utilizada por novas aplicações sem causar congestionamento.

O processo iniciou-se com a transferência do código-fonte para ambas as máquinas Linux (Cliente e Servidor) através da clonagem do respetivo repositório no GitHub:



```
(kali@kali)-[~]
$ git clone https://github.com/jean2/pathload
```

Figura 1.11: Clonagem do repositório do Pathload

Concluída a transferência, procedeu-se à etapa de compilação do programa. Tendo em conta a antiguidade do código do *Pathload*, o compilador moderno (*gcc*) presente no sistema operativo atual abortou a verificação padrão de sintaxe. Para contornar este obstáculo e permitir a geração dos binários, foi necessário recorrer a um método alternativo, injetando *flags* de tolerância (ignorando avisos de variáveis implícitas e tipos de ponteiros) durante o passo de configuração, executando de seguida o comando *make*:

```
(kali@kali)~/pathload
$ CFLAGS="-Wno-implicit-int -Wno-implicit-function-declaration -Wno-incompatible-pointer-types" ./configure
```

Figura 1.12: Configuração do Pathload com flags de compatibilidade do compilador

```
(kali@kali)~/pathload
$ make
```

Figura 1.13: Execução do comando *make* e geração dos binários executáveis

Tendo as ferramentas operacionais de ambos os lados da ligação (*pathload\_snd* e *pathload\_rcv*), encontram-se reunidas as condições para o início das medições.

### 1.3.2 Execução de Testes

#### Cenário Sem Carga

No primeiro teste, a ferramenta foi executada num ambiente de rede isolado, sem qualquer transferência de ficheiros em paralelo.

```
(kali@kali)~/pathload
$ ./pathload_rcv -s 192.168.10.10

Receiver kali starts measurements at sender 192.168.10.10 on Tue Mar 17 13:50:10 2026
Receiving Fleet 0, Rate 43.89Mbps
Receiving Fleet 1, Rate 30.02Mbps
Receiving Fleet 2, Rate 20.73Mbps
Receiving Fleet 3, Rate 16.00Mbps
Receiving Fleet 4, Rate 7.83Mbps
Receiving Fleet 5, Rate 14.95Mbps
Receiving Fleet 6, Rate 7.41Mbps
Receiving Fleet 7, Rate 14.48Mbps
Receiving Fleet 8, Rate 14.36Mbps
Receiving Fleet 9, Rate 7.30Mbps
Receiving Fleet 10, Rate 14.03Mbps
Receiving Fleet 11, Rate 7.54Mbps
Receiving Fleet 12, Rate 13.92Mbps
Receiving Fleet 13, Rate 14.95Mbps
Receiving Fleet 14, Rate 7.83Mbps
Receiving Fleet 15, Rate 14.95Mbps
Receiving Fleet 16, Rate 15.46Mbps
Receiving Fleet 17, Rate 6.99Mbps
Receiving Fleet 18, Rate 12.85Mbps
Receiving Fleet 19, Rate 13.51Mbps
Receiving Fleet 20, Rate 14.71Mbps
Receiving Fleet 21, Rate 14.71Mbps
Receiving Fleet 22, Rate 14.25Mbps
Receiving Fleet 23, Rate 7.02Mbps
Receiving Fleet 24, Rate 13.22Mbps
Receiving Fleet 25, Rate 13.82Mbps
Receiving Fleet 26, Rate 14.25Mbps
Receiving Fleet 27, Rate 14.03Mbps
Receiving Fleet 28, Rate 7.66Mbps

***** RESULT *****
Measurement terminated due to frequent CS @ sender/receiver.
Overhead : 7373200 bytes (7200 KB)
```

Figura 1.14: Resultado do teste Pathload sem tráfego concorrente

### Observação dos Resultados:

Durante a medição, a aplicação enviou frotas de pacotes com taxas compreendidas entre os **6.99 Mbps e os 43.89 Mbps**. No entanto, o teste foi interrompido antes de apresentar uma estimativa final, exibindo a mensagem: *“Measurement terminated due to frequent CS @ sender/receiver”*.

Este erro de *Context Switches* (CS) indica que o processador da máquina virtual sofreu interrupções frequentes que impediram a precisão necessária para o cálculo da largura de banda. Nestas condições, a ferramenta gerou um tráfego de controlo de aproximadamente **7.2 MB (7200 KB)**.

### Cenário Com Carga

Para o segundo cenário, executou-se o teste enquanto decorria em segundo plano uma transferência massiva de dados, simulando uma rede em utilização intensiva.

```
(kali@kali)-[~/pathload]
$ ./pathload_rcv -s 192.168.10.10

Receiver kali starts measurements at sender 192.168.10.10 on Tue Mar 17 14:15:12 2026
Receiving Fleet 0, Rate 14.25Mbps
Receiving Fleet 1, Rate 7.33Mbps

***** RESULT *****
Measurement terminated due to frequent CS @ sender/receiver.
Overhead : 553600 bytes (540 KB)
```

Figura 1.15: Resultado do teste Pathload sob carga

### Observação dos Resultados:

Com a rede sob carga, o impacto no desempenho foi imediato. A ferramenta tentou iniciar as medições com taxas de **14.25 Mbps e 7.33 Mbps**, mas o processo foi abortado logo na segunda frota de pacotes.

A sobrecarga causada pela transferência do ficheiro SCP aumentou a instabilidade do CPU, resultando num erro de *Context Switches* muito mais precoce do que no teste anterior. Devido a este encerramento rápido, o tráfego de controlo gerado foi de apenas **540 KB**.

### Comparação dos Testes e Conclusão

A tabela seguinte resume as métricas obtidas nos dois cenários de teste:

| Cenário   | Taxa Mínima | Taxa Máxima | Resultado     | Overhead |
|-----------|-------------|-------------|---------------|----------|
| Sem Carga | 6.99 Mbps   | 43.89 Mbps  | Abortado (CS) | 7.2 MB   |
| Com Carga | 7.33 Mbps   | 14.25 Mbps  | Abortado (CS) | 0.54 MB  |

Tabela 1.2: Comparação dos resultados obtidos com o Pathload

Em conclusão, os testes demonstram que o *Pathload* é extremamente sensível ao ambiente de virtualização. A dependência da ferramenta em medições de tempo precisas torna a sua execução instável em máquinas virtuais, onde as interrupções do CPU (*Context Switches*) são frequentes. A introdução de



carga na rede acentuou esta limitação, impedindo a obtenção de uma estimativa numérica da largura de banda disponível, ao contrário do que foi possível observar com o *Pathrate*, que se revelou mais resiliente nestas condições.



# Exercício 2: Emulação/Geração De Tráfego De Diferentes Tipos De Aplicações

## Enunciado

Ferramentas para gerar tráfego de rede são importantes e necessárias para realizar testes e avaliar a performance de aplicações ou gestão de recursos numa rede. Neste sentido, o objetivo desta parte passa por realizar alguns testes com algumas aplicações de geração de tráfego, com características próximas àquelas apresentadas por aplicações bem conhecidas (servidor HTTP, SIP, ...) e analisar os resultados.

<https://danielmiessler.com/study/tcpdump/>

<https://userguide.ostinato.org/Quickstart.html>

## 2.1 Exercício 2.1

### Enunciado

Usando a aplicação Ostinato, execute o teste presente no artigo cujo *link* é apresentado em baixo. Use a ferramenta `tcpdump` para armazenar informação sobre o tráfego gerado num ficheiro “`textX.cap`”. Depois de terminado o teste, use a ferramenta de análise de tráfego `tcptrace` para uma leitura geral do resultado.

[https://www.researchgate.net/profile/Anil\\_Gupta41/publication/290304763\\_Evaluation\\_of\\_traffic\\_generators\\_over\\_a\\_40Gbps\\_link/links/5704fe5a08ae13eb88b9385a/Evaluation-of-traffic-generators-over-a-40Gbps-link.pdf](https://www.researchgate.net/profile/Anil_Gupta41/publication/290304763_Evaluation_of_traffic_generators_over_a_40Gbps_link/links/5704fe5a08ae13eb88b9385a/Evaluation-of-traffic-generators-over-a-40Gbps-link.pdf)

### 2.1.1 Introdução

Para este exercício, iremos utilizar a ferramenta Ostinato. O Ostinato é uma aplicação de criação e análise de tráfego de rede, muito utilizada para testes e simulações, sendo extremamente útil para avaliar o desempenho, fazer depuração de redes e testar a segurança informática. A instalação da ferramenta é bastante simples, bastando executar o seguinte comando no terminal:

```
sudo apt install ostinato
```

Para iniciar a aplicação, basta escrever `sudo ostinato` no terminal.

A aplicação apresenta uma interface gráfica intuitiva que facilita a sua configuração. Neste exercício, iremos replicar o teste de *stress* proposto no artigo fornecido no enunciado, utilizando os parâmetros base da figura abaixo:

TABLE 1. packETH parameters

|                       |               |
|-----------------------|---------------|
| Bandwidth             | Maximum       |
| Number of packet sent | Infinite      |
| Protocol              | TCP and UDP   |
| Payload size          | 64-8950 Bytes |
| Experiment time       | 10 seconds    |

Figura 2.1: Tabela de parâmetros do gerador de tráfego (packETH/Ostinato)

### 2.1.2 Configuração do Teste

Para replicar o cenário, os seguintes campos foram preenchidos no separador de configuração da *Stream* no Ostinato:

- **Bandwidth:** Maximum.

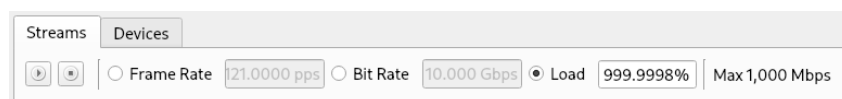


Figura 2.2: Configuração da largura de banda no Ostinato

- **Number of packets sent:** Infinite.

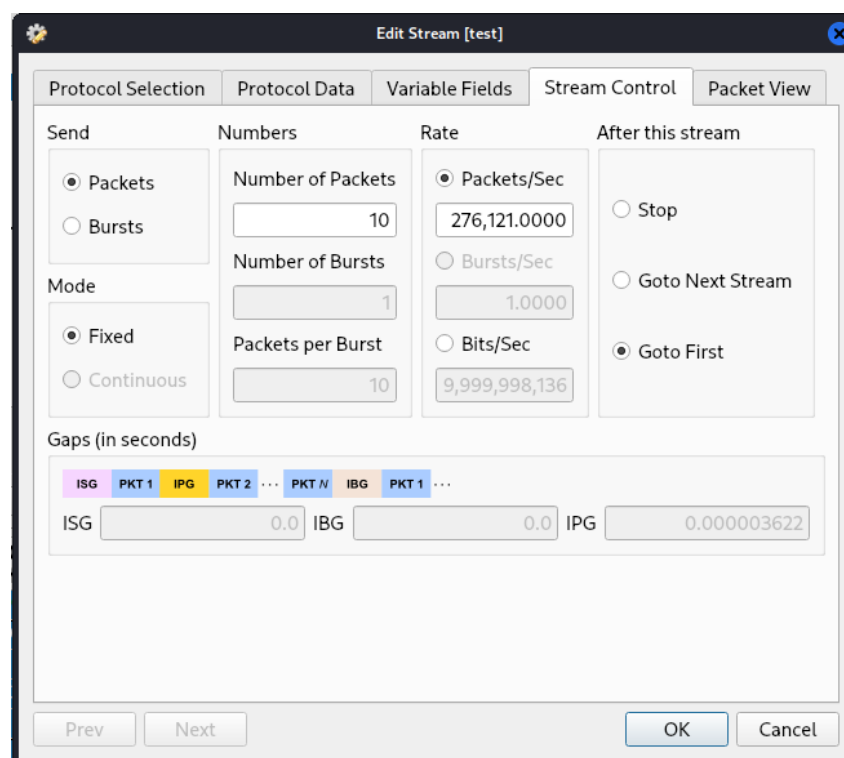


Figura 2.3: Configuração do envio em ciclo infinito

- **Protocol:** TCP (Ethernet II - IPv4 - TCP) e **Payload Size:** 64 a 8950 Bytes.

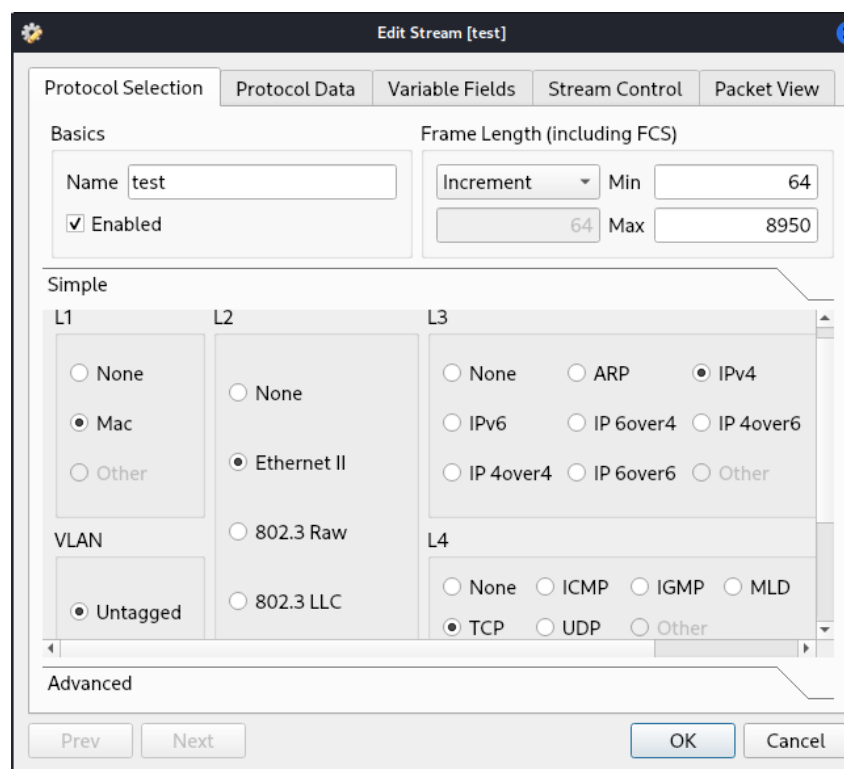


Figura 2.4: Configuração dos protocolos e tamanho do payload

- **Experiment Size:** 10s

### 2.1.3 Metodologia do Teste

Para realizar o teste, utilizámos duas máquinas virtuais na mesma rede (Máquina Cliente com o IP 192.168.10.20 e Máquina Servidor com o IP 192.168.10.10). Para garantir a entrega física dos pacotes neste ambiente virtualizado, o *MAC Address* de destino no Ostinato foi configurado como endereço de *broadcast* (ff:ff:ff:ff:ff:ff).

Na máquina destino (Servidor), iniciámos a captura de tráfego na interface de rede recorrendo ao seguinte comando:



```
(machine2@machine2)-[~]  
$ sudo tcpdump -i eth0 -w testeTCP.pcap
```

Figura 2.5: Comando de inicialização da captura de pacotes com o tcpdump

Com o servidor em modo de escuta, a transmissão massiva de pacotes foi iniciada na máquina Cliente através do Ostinato e interrompida após aproximadamente 10 segundos. Por fim, para processar os resultados guardados no ficheiro `.pcap`, utilizámos a ferramenta `tcptrace`:



```
(machine2@machine2)-[~]  
$ tcptrace -l testeTCP.pcap
```

Figura 2.6: Comando de análise do ficheiro de captura com o tcptrace

### 2.1.4 Resultados Obtidos

A execução do comando `tcptrace` gerou um resumo estatístico detalhado sobre o ficheiro de captura, conforme se pode observar na figura abaixo:

```

(machine2@machine2)~[~]
$ tcptrace -l testeTCP.pcap
1 arg remaining, starting with 'testeTCP.pcap'
Ostermann's tcptrace -- version 6.6.7 -- Thu Nov  4, 2004

60961 packets seen, 60961 TCP packets traced
elapsed wallclock time: 0:00:00.016985, 3589108 pkts/sec analyzed
trace file elapsed time: 0:00:12.261113
TCP connection info:
1 TCP connection traced:
TCP connection 1:
  host a:      192.168.10.20:65535
  host b:      192.168.10.10:65535
  complete conn: no      (SYNs: 0) (FINs: 0)
  first packet: Wed Mar 18 16:35:33.709474 2026
  last packet:  Wed Mar 18 16:35:45.970588 2026
  elapsed time: 0:00:12.261113
  total packets: 60961
  filename:     testeTCP.pcap

a→b:      b→a:
total packets:      60961      total packets:      0
ack pkts sent:      0          ack pkts sent:      0
pure acks sent:      0          pure acks sent:      0
sack pkts sent:      0          sack pkts sent:      0
dsack pkts sent:      0          dsack pkts sent:      0
max sack blks/ack:    0          max sack blks/ack:    0
unique bytes sent:    0          unique bytes sent:    0
actual data pkts:     0          actual data pkts:     0
actual data bytes:    0          actual data bytes:    0
rexmt data pkts:      0          rexmt data pkts:      0
rexmt data bytes:     0          rexmt data bytes:     0
zwnd probe pkts:      0          zwnd probe pkts:      0
zwnd probe bytes:     0          zwnd probe bytes:     0
outoforder pkts:      0          outoforder pkts:      0
pushed data pkts:     0          pushed data pkts:     0
SYN/FIN pkts sent:    0/0        SYN/FIN pkts sent:    0/0
urgent data pkts:      0 pkts    urgent data pkts:      0 pkts
urgent data bytes:     0 bytes    urgent data bytes:     0 byte
mss requested:         0 bytes    mss requested:         0 byte
max segm size:         0 bytes    max segm size:         0 byte

min segm size:         0 bytes    min segm size:         0 byte
avg segm size:         0 bytes    avg segm size:         0 byte
max win adv:           0 bytes    max win adv:           0 byte
min win adv:           0 bytes    min win adv:           0 byte
zero win adv:          0 times    zero win adv:          0 time
avg win adv:           0 bytes    avg win adv:           0 byte
initial window:        0 bytes    initial window:        0 byte
initial window:        0 pkts    initial window:        0 pkts
ttl stream length:     NA         ttl stream length:     NA
missed data:           NA         missed data:           NA
truncated data:        0 bytes    truncated data:        0 byte
truncated packets:     0 pkts    truncated packets:     0 pkts
data xmit time:        0.000 secs  data xmit time:        0.000 secs
idletime max:          5.3 ms      idletime max:          NA ms
hardware dups:         60960 segs  hardware dups:         0 segs

** WARNING: presence of hardware duplicates makes these figures suspec
t!
throughput:            0 Bps      throughput:            0 Bps

```

Figura 2.7: Resultados estatísticos da captura de tráfego (tcptrace)

## Análise dos Resultados

Através da leitura dos resultados fornecidos pelo **tcptrace**, podemos retirar as seguintes conclusões acerca do comportamento da rede e da ferramenta Ostinato:

- **Volume de Tráfego Massivo:** O teste provou ser altamente eficaz na sua capacidade de sobrecarregar a rede, tendo o **tcpdump** registado **60.961 pacotes** enviados do Host A (Cliente) para o Host B (Servidor) num espaço temporal de sensivelmente 12 segundos.
- **Ausência de *Handshake* TCP:** Como o Ostinato atua estritamente como um gerador bruto de tráfego para simulações e testes de *stress*, ele não respeita a arquitetura tradicional de uma ligação TCP. Ou seja, não envia pacotes SYN para iniciar a sessão, nem pacotes FIN para a terminar. Limita-se a injetar as tramas na rede.
- **Respostas do Servidor a Zero (b→a: 0):** Pelo facto de as tramas geradas não constituírem sessões TCP autênticas, a placa de rede do Servidor recebeu o tráfego a nível físico (daí o registo no **tcpdump**), mas o Sistema Operativo descartou a totalidade dos pacotes por serem considerados tráfego inválido, não respondendo com qualquer fluxo de volta ao Cliente.
- **Pacotes Clonados:** Uma vez que configurámos o Ostinato para executar em modo contínuo, o programa entrou em *loop* e enviou exatamente o mesmo pacote ininterruptamente. A ferramenta de análise identificou, e de forma correta, que dos 60.961 pacotes capturados, 60.960 eram cópias exatas do primeiro pacote emitido.

Para evidenciar o comportamento unidirecional e assimétrico deste teste de *stress*, a Tabela 2.1 resume as principais métricas de comunicação entre as duas máquinas.

| Métrica Analisada (tcptrace)                  | Host A → Host B (Cliente) | Host B → Host A (Servidor) |
|-----------------------------------------------|---------------------------|----------------------------|
| Total de Pacotes                              | 60.961                    | 0                          |
| Pacotes de Dados Reais ( <i>Actual Data</i> ) | 0                         | 0                          |
| Pacotes de <i>Handshake</i> (SYN/FIN)         | 0/0                       | 0/0                        |
| Pacotes Duplicados ( <i>Hardware Dups</i> )   | 60.960                    | 0                          |
| <i>Throughput</i> Registado                   | 0 Bps                     | 0 Bps                      |

Tabela 2.1: Comparação do fluxo de tráfego entre a máquina de origem e destino

Em suma, a experiência cumpriu rigorosamente o requisito proposto no artigo, comprovando a eficácia do Ostinato em inundar um canal de rede no limite da sua largura de banda, desconsiderando o estado ou validade do protocolo nas camadas superiores do modelo OSI.

## 2.2 Exercício 2.2

### Enunciado

Experimente outras formas de gerar tráfego com a ferramenta Ostinato (SIP, ...).

## 2.2.1 Introdução

Neste exercício, iremos explorar diferentes formas de gerar tráfego utilizando o Ostinato, com foco em dois cenários distintos: tráfego SIP (Session Initiation Protocol), e a simulação de um ataque DDoS (Distributed Denial of Service). O SIP (Session Initiation Protocol) é um protocolo de sinalização utilizado para estabelecer, modificar e encerrar sessões de comunicação multimédia, como chamadas de voz e vídeo sobre IP (VoIP). Um DDoS (Distributed Denial of Service) é um ataque cibernético que visa sobrecarregar um sistema, servidor ou rede, tornando-o inacessível para utilizadores legítimos. Isso é feito através do envio massivo de tráfego ou requisições simultâneas a partir de múltiplos dispositivos comprometidos (botnets). Existem diferentes tipos de ataques DDoS, como SYN Flood, ICMP Flood (Ping Flood) e UDP Flood, cada um explorando vulnerabilidades específicas nos protocolos de rede.

## 2.2.2 SIP

O Ostinato não tem suporte nativo para SIP. Para contornar esse obstáculo, para simular tráfego SIP vamos utilizar pacotes UDP na porta correta, já que o SIP geralmente usa UDP na porta 5060.

### Configuração do Ostinato

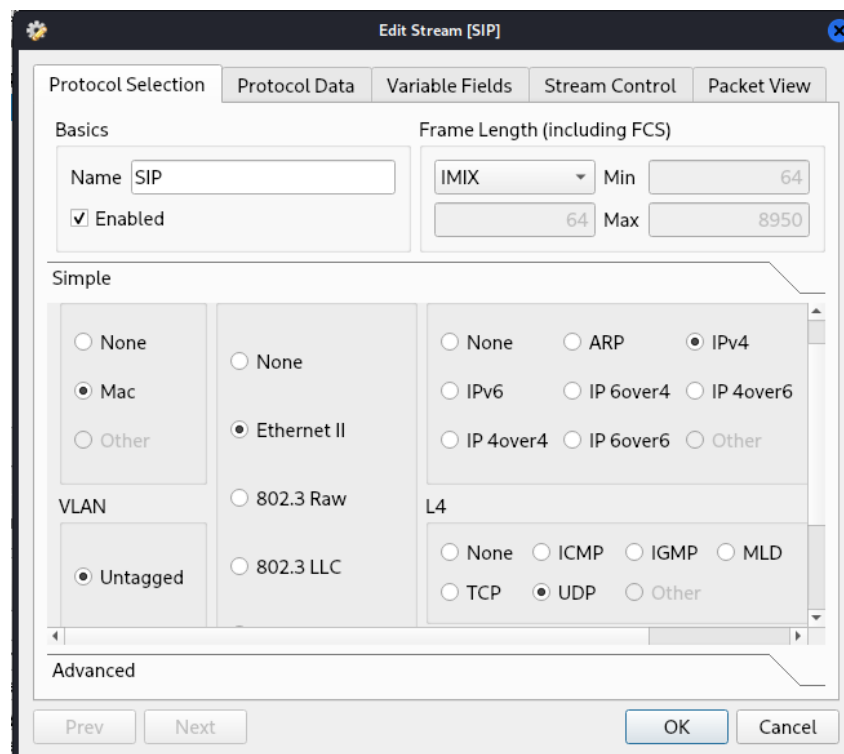


Figura 2.8: Protocol Selection(SIP)

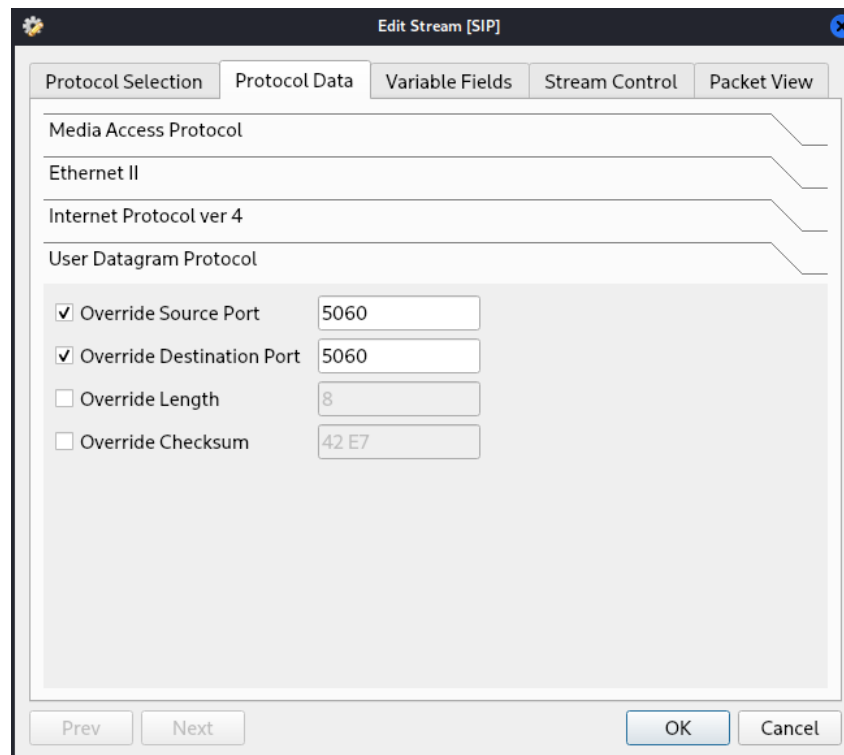


Figura 2.9: Protocol Selectio(SIP)

NOTA: A opção IMIX (Internet Mix) refere-se a um conjunto de tamanhos de pacotes que simulam tráfego real da Internet.

### Realização dos testes

Depois de estar tudo configurado basta começar a simulação no ostinato. Para realizar a captura dos pacotes no servidor vamos por a escutar a interface eth0 que é por onde estão a entrar os dados, para isso usamos o comando:

```
(machine2@machine2)-[~]
$ sudo tcpdump -i eth0 -w testeSIP.pcap
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C99255 packets captured
105520 packets received by filter
2023 packets dropped by kernel
```

Figura 2.10: Escutar a interface eth0

Para haver termos de comparações o teste será de apenas 10s, como não existe nenhuma opção de só executar o teste durante o tempo, será cancelado manualmente passado 10 segundos. Posteriormente serão analisados os resultados com o comando:



```
(machine2@machine2)-[~]
$ tshark -r testeSIP.pcap -qz io,stat,10

IO Statistics
Duration: 18.999598 secs
Interval: 10 secs
Col 1: Frames and bytes

Interval | 1
Interval | Frames | Bytes
0 < 10 | 54383 | 19816782
10 < Dur | 44872 | 16352732
```

Figura 2.11: Resultados Obtidos no Teste SIP

### Observação dos resultados

Duração do teste: 18,99s

Intervalos de análise: 10s

| Intervalo de Tempo (s) | Pacotes Transmitidos | Total de Bytes |
|------------------------|----------------------|----------------|
| 0 ate 10               | 54383                | 19816782       |
| 10 ate 18,99           | 44872                | 16352732       |

Tabela 2.2: Tráfego gerado por intervalo de tempo

### 2.2.3 DDoS

Existem diversas tipologias de ataques **DDoS (Distributed Denial of Service)**, tais como o **SYN Flood** (Ataque TCP), o **ICMP Flood** (Ping Flood) e o **UDP Flood**. De forma a simular um cenário de ataque de maior complexidade, serão criadas e executadas **múltiplas streams em simultâneo**.

#### Preparação

##### SYN Flood

O ataque SYN Flood opera através do **envio massivo de pacotes SYN sem a respetiva conclusão do handshake TCP**, culminando na **exaustão dos recursos** do sistema alvo.

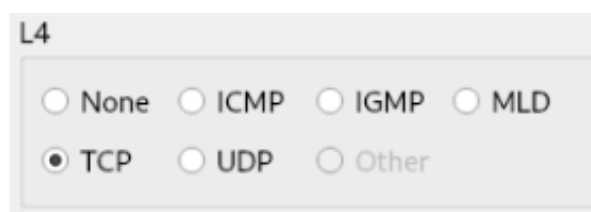


Figura 2.12: Configuração da camada de transporte (L4) para simulação de SYN Flood no Ostinato.

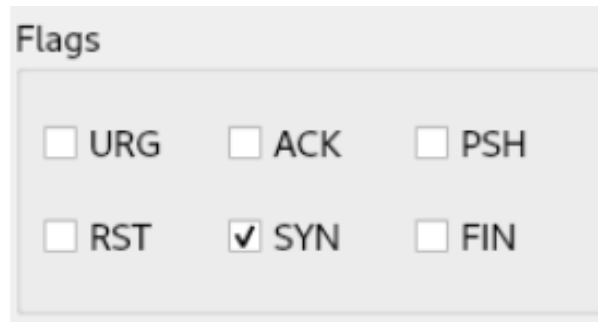


Figura 2.13: Configuração das flags TCP para envio de pacotes SYN.

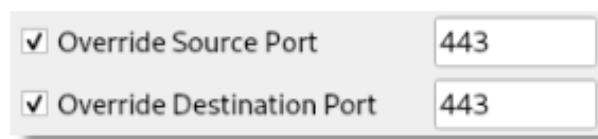


Figura 2.14: Configuração de sobreposição (Override) de parâmetros para o SYN Flood.

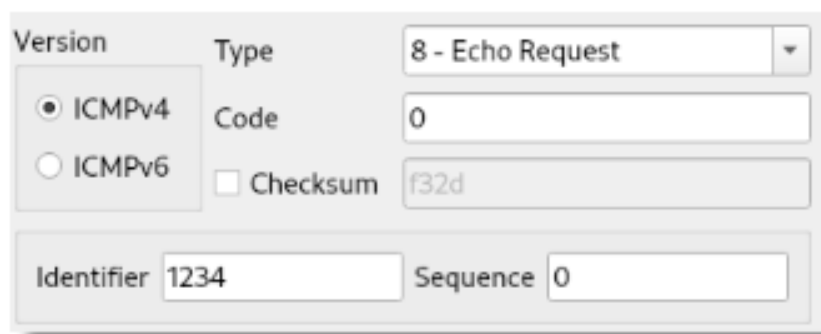
A **seleção da porta 443** justifica-se pelo facto de a esmagadora maioria dos serviços web contemporâneos recorrer ao protocolo **HTTPS**. Consequentemente, um ataque direccionado a este porto pode comprometer diretamente websites, APIs e servidores salvaguardados por protocolos de segurança SSL/-TLS.

## ICMP Flood

Esta modalidade de ataque baseia-se na **emissão de um volume considerável de pacotes ICMP Echo Request** (vulgarmente designados por pings) em direção a um alvo, provocando a **sobrecarga da infraestrutura** de rede.



Figura 2.15: Configuração da camada de rede para o protocolo ICMP no Ostinato.

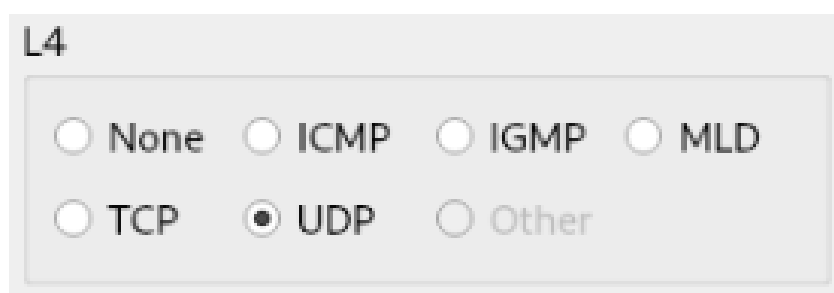


The screenshot shows a configuration window for an ICMP message. It has two tabs: 'Version' and 'Type'. The 'Type' tab is selected, showing a dropdown menu set to '8 - Echo Request'. Below this, there is a 'Code' field set to '0'. A 'Checksum' checkbox is present but unchecked, with a value of 'f32d' shown next to it. At the bottom, there are two fields: 'Identifier' set to '1234' and 'Sequence' set to '0'.

Figura 2.16: Configuração da mensagem ICMP do tipo Echo Request.

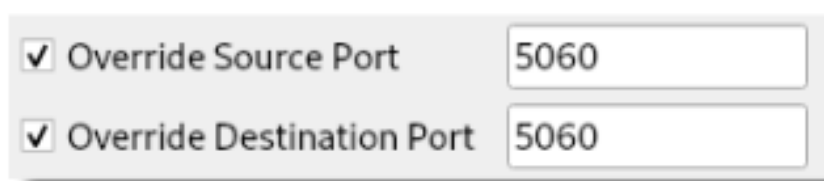
## UDP Flood

Este ataque caracteriza-se pelo **envio massivo de pacotes UDP para um porto específico**, com o intuito de sobrecarregar e indisponibilizar serviços dependentes deste protocolo, tais como sistemas DNS, plataformas VoIP (SIP) ou servidores de jogos online.



The screenshot shows a configuration window titled 'L4'. It contains two rows of radio button options. The first row has 'None', 'ICMP', 'IGMP', and 'MLD'. The second row has 'TCP', 'UDP', and 'Other'. The 'UDP' option in the second row is selected, indicated by a filled circle.

Figura 2.17: Configuração da camada de transporte para o protocolo UDP no Ostinato.



The screenshot shows a configuration window with two checked checkboxes. The first is 'Override Source Port' with a value of '5060' in the adjacent field. The second is 'Override Destination Port' also with a value of '5060' in its adjacent field.

Figura 2.18: Configuração de sobreposição (Override) de parâmetros para o UDP Flood.

## Metodologia dos testes

Com o objetivo de **maximizar o impacto** na rede, recorrer-se-á a **três máquinas distintas**, estando cada uma incumbida da transmissão de um tipo específico de pacote. Esta abordagem visa gerar uma sobrecarga de rede significativamente superior àquela que seria obtida com a utilização de apenas uma máquina a emitir as três tipologias de tráfego. As máquinas emissoras apresentarão os seguintes endereços IP:

- **Cliente 1:** 192.168.1.40 (**SYN Flood**)

- **Ciente 2:** 192.168.1.30 (**ICMP Flood**)
- **Ciente 3:** 192.168.1.20 (**UDP Flood**)

Durante a emissão de pacotes por parte de todas as máquinas envolvidas, proceder-se-á à captura do tráfego recebido na interface **eth0**, recorrendo à seguinte instrução **tcpdump**:

```
(kali@kali)-[~/Desktop]
$ sudo tcpdump -i eth0 -w DDOS.pcap
```

Figura 2.19: Comando de captura de tráfego na interface eth0 utilizando o tcpdump.

Para efeitos de comparação com o ensaio efetuado previamente, a presente execução será igualmente **interrompida de forma manual após o decurso de 10 segundos**. A análise subsequente será conduzida com recurso aos seguintes comandos **tshark**:

```
(kali@kali)-[~/Desktop]
$ tshark -r DDOS.pcap -qz io,stat,10
```

Figura 2.20: Comando de visualização de estatísticas de pacotes capturados com o tshark.

Este comando permite a visualização de **estatísticas** relativas ao volume de pacotes capturados durante o período de testes. Já a instrução:

```
(kali@kali)-[~/Desktop]
$ tshark -r DDOS.pcap
```

Figura 2.21: Comando de análise detalhada do ficheiro pcap gerado.

Destina-se à **extração e análise detalhada** da informação contida em todos os pacotes recolhidos.

## Resultados Obtidos

```
(kali@kali)-[~/Desktop]
$ tshark -r DDOS.pcap -qz io,stat,10
Warning: program compiled against libxml 212 using older 209

=====
| IO Statistics |
| Duration: 10.452627 secs |
| Interval: 10 secs |
| Col 1: Frames and bytes |
|-----|
| Interval | Frames | Bytes |
|-----|
| 0 <-> 10 | 257747 | 61937684 |
| 10 <-> Dur | 12047 | 2791564 |
|-----|
```

Figura 2.22: Estatísticas globais do tráfego capturado durante o ataque DDoS.

```

(kali@kali) ~/Desktop
$ tshark -r 0005.pcap
Warning: program compiled against libxml 212 using older 209
 1 0.000000 192.168.1.10 -> 192.168.1.30 ICMP 60 Echo (ping) reply id=0x04d2, seq=0/0, ttl=64
 2 0.000069 192.168.1.10 -> 192.168.1.30 ICMP 60 Echo (ping) reply id=0x04d2, seq=0/0, ttl=64
 3 0.000141 192.168.1.10 -> 192.168.1.30 ICMP 1514 Echo (ping) reply id=0x04d2, seq=0/0, ttl=64
 4 0.000216 192.168.1.40 -> 192.168.1.10 SSL 60 Continuation Data
 5 0.000217 192.168.1.20 -> 192.168.1.10 UDP 60 5060 -> 5060 Len=18
 6 0.000217 192.168.1.30 -> 192.168.1.10 ICMP 60 Echo (ping) request id=0x04d2, seq=0/0, ttl=127
 7 0.000217 192.168.1.40 -> 192.168.1.10 TCP 60 [TCP Retransmission] 443 -> 443 [SYN] Seq=0 Win=1024 Len=6
 8 0.000218 192.168.1.30 -> 192.168.1.10 ICMP 60 Echo (ping) request id=0x04d2, seq=0/0, ttl=127
 9 0.000218 192.168.1.30 -> 192.168.1.10 ICMP 590 Echo (ping) request id=0x04d2, seq=0/0, ttl=127
10 0.000218 192.168.1.40 -> 192.168.1.10 TCP 60 [TCP Retransmission] 443 -> 443 [SYN] Seq=0 Win=1024 Len=6
11 0.000219 192.168.1.30 -> 192.168.1.10 ICMP 60 Echo (ping) request id=0x04d2, seq=0/0, ttl=127
12 0.000226 192.168.1.10 -> 192.168.1.40 TCP 54 443 -> 443 [RST, ACK] Seq=1 Ack=7 Win=0 Len=0
13 0.000297 192.168.1.10 -> 192.168.1.30 ICMP 60 Echo (ping) reply id=0x04d2, seq=0/0, ttl=64 (request in 11)
14 0.000363 192.168.1.10 -> 192.168.1.40 TCP 54 443 -> 443 [RST, ACK] Seq=1 Ack=7 Win=0 Len=0
15 0.000446 192.168.1.10 -> 192.168.1.30 ICMP 60 Echo (ping) reply id=0x04d2, seq=0/0, ttl=64
16 0.000514 192.168.1.10 -> 192.168.1.30 ICMP 590 Echo (ping) reply id=0x04d2, seq=0/0, ttl=64
17 0.000580 192.168.1.10 -> 192.168.1.40 TCP 54 443 -> 443 [RST, ACK] Seq=1 Ack=7 Win=0 Len=0
18 0.000646 192.168.1.10 -> 192.168.1.30 ICMP 60 Echo (ping) reply id=0x04d2, seq=0/0, ttl=64
19 0.000830 192.168.1.20 -> 192.168.1.10 UDP 60 5060 -> 5060 Len=18
20 0.000831 192.168.1.20 -> 192.168.1.10 UDP 60 5060 -> 5060 Len=18
21 0.000832 192.168.1.30 -> 192.168.1.10 ICMP 1514 Echo (ping) request id=0x04d2, seq=0/0, ttl=127
22 0.000832 192.168.1.40 -> 192.168.1.10 SSL 60 [TCP Port numbers reused] , Continuation Data
23 0.000833 192.168.1.20 -> 192.168.1.10 UDP 590 5060 -> 5060 Len=540
24 0.000833 192.168.1.40 -> 192.168.1.10 TCP 60 [TCP Retransmission] 443 -> 443 [SYN] Seq=0 Win=1024 Len=6
25 0.000834 192.168.1.30 -> 192.168.1.10 ICMP 60 Echo (ping) request id=0x04d2, seq=0/0, ttl=127
26 0.000834 192.168.1.20 -> 192.168.1.10 UDP 60 5060 -> 5060 Len=18
27 0.000855 192.168.1.10 -> 192.168.1.30 ICMP 1514 Echo (ping) reply id=0x04d2, seq=0/0, ttl=64 (request in 25)
28 0.000926 192.168.1.10 -> 192.168.1.40 TCP 54 443 -> 443 [RST, ACK] Seq=1 Ack=7 Win=0 Len=0
29 0.000999 192.168.1.10 -> 192.168.1.40 TCP 54 443 -> 443 [RST, ACK] Seq=1 Ack=7 Win=0 Len=0
30 0.001068 192.168.1.10 -> 192.168.1.30 ICMP 60 Echo (ping) reply id=0x04d2, seq=0/0, ttl=64
31 0.001162 192.168.1.40 -> 192.168.1.10 SSL 590 [TCP Port numbers reused] , Continuation Data
32 0.001163 192.168.1.30 -> 192.168.1.10 ICMP 60 Echo (ping) request id=0x04d2, seq=0/0, ttl=127
33 0.001163 192.168.1.40 -> 192.168.1.10 TCP 60 [TCP Retransmission] 443 -> 443 [SYN] Seq=0 Win=1024 Len=6
34 0.001164 192.168.1.30 -> 192.168.1.10 ICMP 60 Echo (ping) request id=0x04d2, seq=0/0, ttl=127
35 0.001164 192.168.1.40 -> 192.168.1.10 TCP 60 [TCP Retransmission] 443 -> 443 [SYN] Seq=0 Win=1024 Len=6
36 0.001165 192.168.1.30 -> 192.168.1.10 ICMP 590 Echo (ping) request id=0x04d2, seq=0/0, ttl=127
37 0.001165 192.168.1.40 -> 192.168.1.10 TCP 590 [TCP Retransmission] 443 -> 443 [SYN] Seq=0 Win=1024 Len=536
38 0.001165 192.168.1.30 -> 192.168.1.10 ICMP 60 Echo (ping) request id=0x04d2, seq=0/0, ttl=127
39 0.001174 192.168.1.10 -> 192.168.1.40 TCP 54 443 -> 443 [RST, ACK] Seq=1 Ack=537 Win=0 Len=0
40 0.001246 192.168.1.10 -> 192.168.1.30 ICMP 60 Echo (ping) reply id=0x04d2, seq=0/0, ttl=64 (request in 38)
...
90 0.004070 192.168.1.10 -> 192.168.1.30 ICMP 60 Echo (ping) reply id=0x04d2, seq=0/0, ttl=64
91 0.004136 192.168.1.10 -> 192.168.1.40 TCP 54 443 -> 443 [RST, ACK] Seq=1 Ack=537 Win=0 Len=0
92 0.004286 192.168.1.10 -> 192.168.1.40 TCP 54 443 -> 443 [RST, ACK] Seq=1 Ack=7 Win=0 Len=0
93 0.004289 192.168.1.10 -> 192.168.1.30 ICMP 60 Echo (ping) reply id=0x04d2, seq=0/0, ttl=64
94 0.004369 192.168.1.10 -> 192.168.1.40 TCP 54 443 -> 443 [RST, ACK] Seq=1 Ack=7 Win=0 Len=0
95 0.004458 192.168.1.40 -> 192.168.1.10 SSL 590 [TCP Port numbers reused] , Continuation Data
96 0.004459 192.168.1.40 -> 192.168.1.10 TCP 60 [TCP Retransmission] 443 -> 443 [SYN] Seq=0 Win=1024 Len=6
97 0.004459 192.168.1.30 -> 192.168.1.10 ICMP 60 Echo (ping) request id=0x04d2, seq=0/0, ttl=127
98 0.004459 192.168.1.40 -> 192.168.1.10 TCP 60 [TCP Retransmission] 443 -> 443 [SYN] Seq=0 Win=1024 Len=6
99 0.004460 192.168.1.30 -> 192.168.1.10 ICMP 60 Echo (ping) request id=0x04d2, seq=0/0, ttl=127
100 0.004460 192.168.1.40 -> 192.168.1.10 TCP 590 [TCP Retransmission] 443 -> 443 [SYN] Seq=0 Win=1024 Len=536
...

```

Figura 2.23: Análise detalhada dos pacotes do ataque DDoS capturados.

## Análise dos Resultados

Duração do teste: 10,45s

Intervalos de análise: 10s

| Intervalo de Tempo (s) | Pacotes Transmitidos | Total de Bytes |
|------------------------|----------------------|----------------|
| 0 ate 10               | 257.747              | 61.937.684     |
| 10 ate 10,47           | 12.047               | 2.791.564      |

Tabela 2.3: Tráfego gerado por intervalo de tempo no ataque DDoS.

## Conclusão

A análise do tráfego capturado, efetuada através da ferramenta tshark, evidenciou que num **curto espaço de tempo** (pouco mais de 10 segundos, correspondente à duração do ensaio), uma simples simulação de ataque foi capaz de gerar um volume de **269 794 pacotes**, perfazendo um total de **64 729 248 Bytes** de tráfego.

Este ensaio empírico corroborou a eficácia e o **impacto substancial** que ataques DDoS de cariz simples podem infligir sobre a disponibilidade da rede. Sublinha-se, assim, a **importância crítica** da implementação de **mecanismos de mitigação e defesa proativa**, nomeadamente firewalls avançadas, políticas de *rate limiting* e sistemas de deteção de anomalias.

## 2.3 Exercício 2.3

### Enunciado

Escolha duas ferramentas de geração de tráfego (de [https://wiki.wireshark.org/Tools#Traffic\\_generators](https://wiki.wireshark.org/Tools#Traffic_generators)) e execute alguns testes, de forma a gerar tráfego semelhante àquele gerado por acesso a servidor *web* e FTP.

### 2.3.1 Introdução

Para a realização destes testes, selecionou-se a ferramenta **Scapy**, sendo esta a única que apresentou resultados consistentes face aos ensaios efetuados.

O **Scapy** consiste numa poderosa ferramenta de manipulação de pacotes de rede, desenvolvida em **Python**, que viabiliza a criação, envio, captura e análise de pacotes em múltiplos protocolos. A sua aplicabilidade estende-se a **testes de segurança, análise forense, simulação de tráfego e engenharia reversa de protocolos**, bem como a atividades de *port scanning* e *fuzzing*.

Para iniciar a ferramenta, basta executar o seguinte comando:



```
(kali@kali)-[~/Desktop]
$ sudo scapy
```

Figura 2.24: Inicialização da interface interativa da ferramenta Scapy.

A operação do Scapy processa-se através do interpretador Python, proporcionando aos utilizadores a capacidade de interagir com a ferramenta mediante o recurso a **funções específicas** para criar, enviar, capturar e manipular pacotes de rede. Adicionalmente, esta plataforma possibilita a **automação de testes e análises** através de *scripts*, revelando-se de extrema utilidade tanto para testes manuais como para execuções automatizadas.

### 2.3.2 Início dos testes

No presente ensaio, recorrer-se-á a uma função basilar do Scapy para a transmissão de **10 pacotes do tipo TCP** com destino ao servidor.

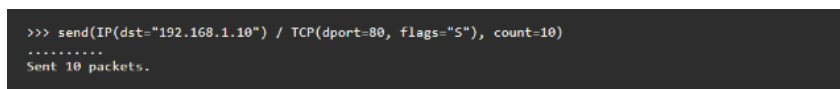
Primeiramente, a interface **eth0** será configurada em **modo de escuta** para possibilitar a captura e posterior análise dos pacotes rececionados.



```
(kali@kali)~[/Desktop]
└─$ sudo tcpdump -i eth0 -w SCAPY.pcap
```

Figura 2.25: Configuração da interface eth0 em modo de escuta no servidor.

Com a interface devidamente configurada, procede-se à execução da função de envio no Scapy, utilizando a seguinte instrução:



```
>>> send(IP(dst="192.168.1.10") / TCP(dport=80, flags="S"), count=10)
.....
Sent 10 packets.
```

Figura 2.26: Instrução do Scapy utilizada no cliente para envio de pacotes.

Esta instrução desencadeia a transmissão sucessiva de **10 pacotes** em direção ao servidor.

#### Explicação da função

A instrução executada decompõe-se nos seguintes elementos:

- **send()**: Função responsável pelo envio de pacotes.
- **IP(dst="192.168.1.10")**: Define a camada de rede com o protocolo **IP**, estabelecendo o endereço de destino do pacote.
- **/**: Operador que permite o empilhamento de diferentes camadas do modelo OSI.
- **TCP(dport=80, flags="S")**: Define a camada de transporte com o protocolo **TCP**, especificando o porto de destino (**80** - associado a tráfego HTTP) e a flag **"S"** (**SYN**), tipicamente utilizada para iniciar uma conexão.
- **count=10**: Parâmetro que estipula o número total de pacotes a enviar.

**Resumo do Comando:** Em suma, este comando gera e transmite **10 pacotes TCP SYN** direcionados ao endereço 192.168.1.10 no porto 80.

### 2.3.3 Resultados Obtidos

```
(kali@kali)-[~/Desktop]
$ tshark -r SCAPY.pcap
Warning: program compiled against libxml 212 using older 209
1 0.000000 PCSSystemtec_98:5c:7f > Broadcast ARP 60 Who has 192.168.1.10? Tell 192.168.1.20
2 0.000018 PCSSystemtec_6e:13:6e > PCSSystemtec_98:5c:7f ARP 42 192.168.1.10 is at 08:00:27:6e:13:6e
3 0.022159 192.168.1.20 > 192.168.1.10 TCP 60 20 > 80 [SYN] Seq=0 Win=8192 Len=0
4 0.022194 192.168.1.10 > 192.168.1.20 TCP 54 80 > 20 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
5 0.022874 192.168.1.20 > 192.168.1.10 TCP 60 [TCP Port numbers reused] 20 > 80 [SYN] Seq=0 Win=8192 Len=0
6 0.022903 192.168.1.10 > 192.168.1.20 TCP 54 80 > 20 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
7 0.023875 192.168.1.20 > 192.168.1.10 TCP 60 [TCP Port numbers reused] 20 > 80 [SYN] Seq=0 Win=8192 Len=0
8 0.023917 192.168.1.10 > 192.168.1.20 TCP 54 80 > 20 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
9 0.026570 192.168.1.20 > 192.168.1.10 TCP 60 [TCP Port numbers reused] 20 > 80 [SYN] Seq=0 Win=8192 Len=0
10 0.026597 192.168.1.10 > 192.168.1.20 TCP 54 80 > 20 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
11 0.028418 192.168.1.20 > 192.168.1.10 TCP 60 [TCP Port numbers reused] 20 > 80 [SYN] Seq=0 Win=8192 Len=0
12 0.028451 192.168.1.10 > 192.168.1.20 TCP 54 80 > 20 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
13 0.028966 192.168.1.20 > 192.168.1.10 TCP 60 [TCP Port numbers reused] 20 > 80 [SYN] Seq=0 Win=8192 Len=0
14 0.028992 192.168.1.10 > 192.168.1.20 TCP 54 80 > 20 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
15 0.030636 192.168.1.20 > 192.168.1.10 TCP 60 [TCP Port numbers reused] 20 > 80 [SYN] Seq=0 Win=8192 Len=0
16 0.030676 192.168.1.10 > 192.168.1.20 TCP 54 80 > 20 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
17 0.032135 192.168.1.20 > 192.168.1.10 TCP 60 [TCP Port numbers reused] 20 > 80 [SYN] Seq=0 Win=8192 Len=0
18 0.032173 192.168.1.10 > 192.168.1.20 TCP 54 80 > 20 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
19 0.032782 192.168.1.20 > 192.168.1.10 TCP 60 [TCP Port numbers reused] 20 > 80 [SYN] Seq=0 Win=8192 Len=0
20 0.033186 192.168.1.10 > 192.168.1.20 TCP 54 80 > 20 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
21 0.033807 192.168.1.20 > 192.168.1.10 TCP 60 [TCP Port numbers reused] 20 > 80 [SYN] Seq=0 Win=8192 Len=0
22 0.033825 192.168.1.10 > 192.168.1.20 TCP 54 80 > 20 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
23 5.123688 PCSSystemtec_6e:13:6e > PCSSystemtec_98:5c:7f ARP 42 Who has 192.168.1.20? Tell 192.168.1.10
24 5.124857 PCSSystemtec_98:5c:7f > PCSSystemtec_6e:13:6e ARP 60 192.168.1.20 is at 08:00:27:98:5c:7f
```

Figura 2.27: Resultados da captura de pacotes efetuada no servidor através do tshark.

```
(kali@kali)-[~/Desktop]
$ tcptrace -l SCAPY.pcap
1 arg remaining, starting with 'SCAPY.pcap'
Ostermann's tcptrace -- version 6.6.7 -- Thu Nov 4, 2004

20 packets seen, 20 TCP packets traced
elapsed wallclock time: 0:00:00.000776, 25773 pkts/sec analyzed
trace file elapsed time: 0:00:00.011666
TCP connection info:
1 TCP connection traced:
TCP connection 1:
  host a:      192.168.1.20:20
  host b:      192.168.1.10:80
  complete conn: RESET (SYNs: 1) (FINS: 0)
  first packet: Sat Mar 8 06:42:31.480746 2025
  last packet:  Sat Mar 8 06:42:31.492412 2025
  elapsed time: 0:00:00.011666
  total packets: 20
  filename:     SCAPY.pcap

a->b:
total packets:      10
resets sent:        0
ack pkts sent:      0
pure acks sent:     0
sack pkts sent:     0
dsack pkts sent:   0
max sack blks/ack:  0
unique bytes sent:  0
actual data pkts:   0
actual data bytes:  0
rexmt data pkts:    9
rexmt data bytes:   9
zwnd probe pkts:    0
zwnd probe bytes:   0
outoforder pkts:    0
pushed data pkts:   0
SYN/FIN pkts sent: 10/0
urgent data pkts:   0 pkts
urgent data bytes:  0 bytes
mss requested:      0 bytes
max segm size:      0 bytes
min segm size:      0 bytes
avg segm size:      0 bytes
max win adv:        8192 bytes
min win adv:        8192 bytes
zero win adv:       0 times
avg win adv:        8192 bytes
initial window:     0 bytes
initial window:     0 pkts
ttl stream length:  NA
missed data:        NA
truncated data:     0 bytes
truncated packets:  0 pkts
data xmit time:     0.000 secs
idletime max:       2.7 ms
throughput:         0 Bps

b->a:
total packets:      10
resets sent:        10
ack pkts sent:      10
pure acks sent:     0
sack pkts sent:     0
dsack pkts sent:   0
max sack blks/ack:  0
unique bytes sent:  0
actual data pkts:   0
actual data bytes:  0
rexmt data pkts:    0
rexmt data bytes:   0
zwnd probe pkts:    0
zwnd probe bytes:   0
outoforder pkts:    0
pushed data pkts:   0
SYN/FIN pkts sent: 0/0
urgent data pkts:   0 pkts
urgent data bytes:  0 bytes
mss requested:      0 bytes
max segm size:      0 bytes
min segm size:      0 bytes
avg segm size:      0 bytes
max win adv:        0 bytes
min win adv:        0 bytes
zero win adv:       0 times
avg win adv:        0 bytes
initial window:     0 bytes
initial window:     0 pkts
ttl stream length:  NA
missed data:        NA
truncated data:     0 bytes
truncated packets:  0 pkts
data xmit time:     0.000 secs
idletime max:       2.7 ms
throughput:         0 Bps
```

Figura 2.28: Análise da conexão TCP gerada, utilizando a ferramenta tcptrace.



### 2.3.4 Análise dos Resultados

- **Resumo da Conexão:**

- A conexão foi interrompida por um **RESET (RST)**, indicando que não foi encerrada de forma convencional através de uma flag FIN.
- Registou-se o envio de **1 pacote SYN** e a total ausência de pacotes FIN, o que corrobora a não concretização de um encerramento normal da sessão.

- **Tempo e Pacotes:**

- Verificou-se a captura de um total de **20 pacotes**, distribuídos equitativamente (**10 pacotes em cada direção**).

- **Análise Direcional:**

- **Fluxo 192.168.1.20 -> 192.168.1.10 (Cliente para Servidor):**
  - \* Transmissão inicial de 10 pacotes TCP.
  - \* Observaram-se **9 pacotes retransmitidos**, sugerindo uma possível perda de pacotes ou a ausência de confirmação de receção (*ACKnowledgment*) adequada.
  - \* **Ausência de transmissão de dados úteis** (*unique bytes sent: 0*).
- **Fluxo 192.168.1.10 -> 192.168.1.20 (Servidor para Cliente):**
  - \* Resposta materializada em 10 pacotes TCP.
  - \* Emissão de **10 pacotes RST**, indício forte de que este *host* recusou a conexão ou que a sessão foi liminarmente considerada inválida.
  - \* **Ausência de envio de dados** (*unique bytes sent: 0*).

### 2.3.5 Conclusão

A análise aos resultados evidencia uma tentativa inicial de estabelecimento de uma conexão TCP, a qual foi **imediatamente recusada** pelo destinatário através da emissão de **pacotes RST**. O emissor procedeu à retransmissão de múltiplos pacotes sem sucesso, verificando-se a **ausência de transmissão de carga útil** (*payload*).

Este comportamento indica que o *host* de destino rejeitou a conexão, um cenário que poderá derivar da ação de uma **firewall**, de um **serviço inativo** no porto especificado (porto 80) ou de uma **rejeição intencional** da comunicação.